

Trabalho 3 — Implantação Multicloud da Aplicação Biblioteca

Disciplina: Cloud Computing e DevOps Avançado — CST em Sistemas para Internet (6º Período)

Professor: Diogo P. Ranghetti

Equipe: Adrian Souza · Fernando Cardoso · Guilherme Vitor · Renan Oliveira · Victor Caitano

Aplicação: <https://github.com/dpRanghetti/biblioteca>

Ambientes escolhidos: Amazon Web Services (EKS) · Microsoft Azure (AKS) · Hetzner Cloud (VPS + K3s)

Tutorial reproduzível: containerização com Docker, implantação em Kubernetes nos três ambientes, comparação de custos e recomendação final.

Sumário

1. Capa e integrantes
 2. Objetivo e escopo
 3. Análise da aplicação Biblioteca
 4. Pré-requisitos
 5. Containerização com Docker
 6. Manifestos Kubernetes
 7. Comandos comuns aos três ambientes
 8. Provedor 1 — AWS (Amazon EKS)
 9. Provedor 2 — Microsoft Azure (AKS)
 10. Provedor 3 — Hetzner Cloud (VPS + K3s)
 11. Evidências e testes
 12. Comparação de custos
 13. Comparação qualitativa — vantagens e desvantagens
 14. Recomendação final
 15. Limpeza dos recursos
 16. Erros comuns e diagnóstico
 17. Checklists finais
 18. Glossário
 19. Referências
-

1. Capa e integrantes

Campo	Valor
Integrantes	Adrian Souza · Fernando Cardoso · Guilherme Vitor · Renan Oliveira · Victor Caitano
Data da elaboração	02/09/2026
Repositório da equipe	https://github.com/Renan0204/t3-biblioteca-multicloud
Versão da aplicação utilizada	<code>dpRanghetti/biblioteca</code> , branch <code>master</code> , commit <code>5a40964</code> (2026-06-05) — confirmar no clone da equipe

2. Objetivo e escopo

Produzir um **tutorial reproduzível** que permita a outra equipe executar a aplicação **Biblioteca** com **Docker** e **Kubernetes** em **três provedores distintos**:

- **AWS** — Amazon Elastic Kubernetes Service (EKS), Kubernetes gerenciado.

- **Microsoft Azure** — Azure Kubernetes Service (AKS), Kubernetes gerenciado.
- **Hetzner Cloud** — VPS Linux com **K3s** (Kubernetes leve de nó único), autogerenciado.

A combinação atende à *Regra dos Três Ambientes* (slide 6): dois entre AWS/GCP/Azure mais um terceiro ambiente diferente, de outro provedor.

Escopo e limitações (slide 11):

- A implantação é **demonstrativa**. O banco é **H2 em memória** (`jdbc:h2:mem:banco`): ao reiniciar o container/Pod, **os dados cadastrados são perdidos**.
- Por isso, usamos **1 réplica** em todos os ambientes.
- Não há migração para banco externo (não é obrigatório — slide 11).
- A **mesma imagem** é usada nos três ambientes; os manifestos só mudam onde o provedor exige (a forma de exposição: `LoadBalancer` na nuvem gerenciada, `NodePort` /Ingress na VPS — slide 12).

Arquitetura mínima (slide 12):

```

Usuário
  |
IP ou DNS público
  |
Service (LoadBalancer) | NodePort / Ingress / proxy reverso
  |
Deployment Kubernetes (replicas: 1)
  |
Pod com container da Biblioteca
  |
H2 em memória dentro do processo
  
```

3. Análise da aplicação Biblioteca

Fatos levantados no `pom.xml` e em `src/main/resources/application.properties`:

Item	Valor
Framework	Spring Boot 4.0.6
Runtime	Java 21 (<code><java.version>21</java.version></code>)
Coordenadas Maven	<code>com.unialfa : biblioteca : 0.0.1-SNAPSHOT</code>
Artefato gerado	<code>target/biblioteca-0.0.1-SNAPSHOT.jar</code>
Build	Maven (com <code>mvnw</code> / <code>mvnw.cmd</code> no repositório)
Porta	8080 (<code>server.port=8080</code>)
Banco	H2 em memória — <code>spring.datasource.url=jdbc:h2:mem:banco</code> (<code>sa</code> / sem senha)
Console H2	habilitado em <code>/h2-console</code>
Interface web	Thymeleaf + Spring Security (login por formulário em <code>/login</code>)
API REST	protegida por JWT — login em <code>POST /api/auth/login</code> , demais rotas com <code>Bearer</code>
Segredo JWT	propriedade <code>api.security.token.secret</code> (HS256, expiração de 24 h)
Usuários iniciais	<code>admin</code> / <code>admin</code> (ROLE_ADMIN) e <code>user</code> / <code>user</code> (ROLE_USER)

Configuração externa e segredos (slides 10, 22):

- O `application.properties` traz um valor de exemplo para `api.security.token.secret` . **Não** usamos esse valor. Cada integrante fornece o seu, por variável de ambiente `API_SECURITY_TOKEN_SECRET` (o Spring Boot faz o *relaxed binding* `API_SECURITY_TOKEN_SECRET` → `api.security.token.secret`).
- No Kubernetes, o valor fica em um objeto `Secret` e **nunca** é escrito no tutorial público, no repositório ou em capturas de tela.
- As credenciais `admin/admin` e `user/user` servem **apenas para o ambiente acadêmico** e não são adequadas para produção (slide 10).

Requisitos de execução para o container:

1. Comando de início: `java -jar biblioteca-0.0.1-SNAPSHOT.jar` .
2. Runtime: JRE 21.
3. Porta exposta: 8080.
4. Variável obrigatória em produção: `API_SECURITY_TOKEN_SECRET` .
5. A aplicação **não** grava dados em disco (H2 em memória) — não precisa de volume.
6. Endpoint usado nas *probes*: `GET /login` (público, responde 200).

4. Pré-requisitos

4.1 Ambiente local (slide 14)

Ferramenta	Verificação
Git	<code>git --version</code>
Docker Engine / Docker Desktop (com BuildKit)	<code>docker version</code>
<code>kubectl</code>	<code>kubectl version --client</code>
Editor/IDE	VS Code, IntelliJ, etc.

4.2 Ferramentas por provedor

Provedor	CLI	Verificação
AWS	<code>aws</code> (v2) + <code>eksctl</code>	<code>aws --version</code> · <code>eksctl version</code>
Azure	<code>az</code>	<code>az version</code>
Hetzner	cliente SSH (+ opcional <code>hcloud</code>)	<code>ssh -V</code> · <code>hcloud version</code>

4.3 Contas e permissões

- Contas ativas na **AWS**, na **Azure** e na **Hetzner Cloud**.
- Permissão para criar clusters, redes, registros de imagem e balanceadores.
- Recomendado: **MFA** habilitado e **orçamento/alertas de custo** configurados (slide 54).

4.4 Conta de registro de imagem

- Conta no **Docker Hub** (usaremos um repositório **público** — aceitável apenas para esta atividade, slide 21). Alternativas por provedor (ECR, ACR) estão nas seções 8 e 9.

5. Containerização com Docker

5.1 Clonar e verificar o projeto (slide 15)

```
git clone https://github.com/dpRanghetti/biblioteca.git
cd biblioteca
git log -1 --format="%H %ci" # registrar o commit/data usados no tutorial
```

Confirme a existência de: `pom.xml`, `mvnw`, `mvnw.cmd`, `src/main/java`,
`src/main/resources/application.properties`.

5.2 Dockerfile (multi-stage)

Copie o arquivo `Dockerfile` deste repositório para a **raiz do projeto** `biblioteca`. Decisões (o enunciado permite melhorar o arquivo do slide 16, desde que explicado — slide 16):

Decisão	Motivo
Build multi-stage (<code>maven</code> → <code>eclipse-temurin:21-jre</code>)	imagem final sem Maven, menor e com menos superfície de ataque
Cache do <code>~/.m2</code> via BuildKit	builds seguintes muito mais rápidos
<code>-DskipTests</code> no build da imagem	testes rodam no pipeline, não na construção da imagem do lab
Usuário <code>spring</code> não-root (<code>USER spring:spring</code>)	boa prática de segurança (Aula 05, slides 25 e 49)
<code>-XX:MaxRAMPercentage=75.0</code>	a JVM respeita o limite de memória do container
<code>EXPOSE 8080</code>	porta padrão do Spring Boot

5.3 .dockerignore

Copie `.dockerignore` para a raiz do projeto. Ele remove `.git`, `target`, `docs`, `*.md`, IDE e arquivos sensíveis do contexto de build (slide 17).

5.4 Construir a imagem (slide 18)

```
docker build -t biblioteca:1.0 .
```

Apple Silicon / Windows ARM: os nós dos três ambientes são **x86_64**. Force a arquitetura: `docker buildx build --platform linux/amd64 -t biblioteca:1.0 .` (senão o Pod falha com `exec format error`).

Verificações obrigatórias:

```
docker image ls biblioteca
docker history biblioteca:1.0
```

[EVIDÊNCIA: saída de `docker image ls` mostrando `biblioteca:1.0` e o tamanho]

5.5 Executar e validar localmente (slides 19–20)

```
# Linux/macOS
docker run --name biblioteca-local -p 8080:8080 \
  -e API_SECURITY_TOKEN_SECRET="$(openssl rand -base64 32)" \
  biblioteca:1.0
```

```
# Windows PowerShell
$b = [byte[]]::new(32); [Security.Cryptography.RandomNumberGenerator]::Fill($b)
docker run --name biblioteca-local -p 8080:8080 -e
API_SECURITY_TOKEN_SECRET=$([Convert]::ToBase64String($b)) biblioteca:1.0
```

Validar:

- Abrir `http://localhost:8080/login` e entrar com `admin` / `admin`.
- Cadastrar um autor e um livro (uma operação de escrita).
- Consultar os logs: `docker logs biblioteca-local`.
- API: `curl -s -X POST http://localhost:8080/api/auth/login -H "Content-Type: application/json" -d '{"login":"admin","senha":"admin"}'` (ajuste os nomes dos campos conforme o `AuthController` do projeto).

[EVIDÊNCIA: tela de login + tela após autenticar + trecho do log de inicialização]

Encerrar o teste local:

```
docker stop biblioteca-local && docker rm biblioteca-local
```

Corrija o container **localmente** antes de ir para a nuvem. Não use o cluster para depurar o Dockerfile (slide 20).

5.6 Publicar a imagem no Docker Hub (slide 21)

Estratégia: **construir uma vez, usar nos três ambientes** (slide 12).

```
docker login
docker tag biblioteca:1.0 docker.io/SEU_USUARIO_DOCKERHUB/biblioteca:1.0
docker push docker.io/SEU_USUARIO_DOCKERHUB/biblioteca:1.0
```

Deixe o repositório **público** para esta atividade (ou mantenha privado e configure `imagePullSecret` — ver seções por provedor).

[EVIDÊNCIA: repositório no Docker Hub com a tag 1.0]

6. Manifestos Kubernetes

Arquivos neste repositório (diretório `k8s/`):

```

k8s/
├─ base/
│   ├── deployment.yaml      # 1 réplica, Secret via env, probes, limites
│   └─ secret.example.yaml   # MODELO - não aplicar direto
├─ managed/
│   └─ service-loadbalancer.yaml # AWS EKS + Azure AKS
└─ vps/
    ├── service-nodeport.yaml # Hetzner + K3s (porta 30080)
    └─ ingress-traefik.yaml   # opcional: Ingress + HTTPS na VPS

```

6.1 Secret (slide 23)

Crie o **Secret** sem gravar o valor em disco:

```

kubectl create secret generic biblioteca-secret \
  --from-literal=jwt-secret="$(openssl rand -base64 32)"

```

O arquivo `k8s/base/secret.example.yaml` documenta o formato e traz a variante para PowerShell. **Nunca** versione o valor real (slides 22 e 54).

6.2 Deployment (slides 24–26)

`k8s/base/deployment.yaml`. Pontos-chave:

- `replicas: 1` (por causa do H2 em memória).
- `image:` — **substitua** por `docker.io/SEU_USUARIO_DOCKERHUB/biblioteca:1.0`.
- `env` `API_SECURITY_TOKEN_SECRET` via `secretKeyRef` → `biblioteca-secret` / `jwt-secret`.
- `resources`: requests `250m` / `384Mi`; limits `1` / `768Mi` (ponto de partida — registrar o valor efetivamente usado; se houver `OOMKilled`, subir para `1Gi`).
- `startupProbe` + `readinessProbe` + `livenessProbe` em `GET /login` (porta 8080).
- `securityContext`: `runAsNonRoot`, `allowPrivilegeEscalation: false`, drop de todas as *capabilities*.

6.3 Service

Ambiente	Arquivo	Tipo	Acesso
AWS EKS / Azure AKS	<code>k8s/managed/service-loadbalancer.yaml</code>	LoadBalancer	balanceador com IP/DNS público (recurso cobrado — slide 27)
Hetzner + K3s	<code>k8s/vps/service-nodeport.yaml</code>	NodePort	<code>http://IP_DA_VPS:30080/login</code>
Hetzner + K3s (opcional)	<code>k8s/vps/ingress-traefik.yaml</code>	Ingress (Traefik)	<code>http(s)://SEU_DOMINIO/</code>

7. Comandos comuns aos três ambientes

Depois que o `kubectl` estiver apontando para o cluster correto (cada seção mostra como), o fluxo é o **mesmo** (slide 28):

```
# 1) Segredo
kubectl create secret generic biblioteca-secret \
  --from-literal=jwt-secret="$(openssl rand -base64 32)"

# 2) Deployment (com a imagem já ajustada no arquivo)
kubectl apply -f k8s/base/deployment.yaml

# 3) Service – escolha conforme o ambiente:
kubectl apply -f k8s/managed/service-loadbalancer.yaml      # AWS / Azure
# ou
kubectl apply -f k8s/vps/service-nodeport.yaml            # Hetzner / K3s

# 4) Acompanhar
kubectl get pods
kubectl get deployment
kubectl get service
kubectl rollout status deployment/biblioteca

# 5) Diagnóstico
kubectl describe pod <NOME_DO_POD>
kubectl logs deployment/biblioteca
```

Verificação de "pronto": `kubectl get pods` deve mostrar `1/1 Running` e `kubectl get endpoints biblioteca` deve listar o IP do Pod na porta 8080.

8. Provedor 1 — AWS (Amazon EKS)

Produto: Amazon EKS · **Região de exemplo:** `us-east-1` (N. Virginia). Roteiro conforme slides 29–30.

8.1 Conta, região e identidade

```
aws configure          # Access Key, Secret, região us-east-1, output json
aws sts get-caller-identity  # confirma a conta/identidade
```

[EVIDÊNCIA: `get-caller-identity` com o Account ID ocultado]

8.2 Registro ECR (alternativa ao Docker Hub)

```
aws ecr create-repository --repository-name biblioteca --region us-east-1

ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
ECR=$ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com

aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS --password-stdin $ECR

docker tag biblioteca:1.0 $ECR/biblioteca:1.0
docker push $ECR/biblioteca:1.0
```

Se usar o ECR, ajuste `image:` no `deployment.yaml` para `$ECR/biblioteca:1.0`. Os nós criados pelo `eksctl` já recebem a política `AmazonEC2ContainerRegistryReadOnly`, então o `pull` do ECR funciona **sem** `imagePullSecret`.

8.3 Criar o cluster EKS e os nós

```
eksctl create cluster \
  --name biblioteca \
  --region us-east-1 \
  --nodes 2 --node-type t3.medium \
  --managed
```

- Demora ~**15–20 min** (o `eksctl` cria uma stack CloudFormation com VPC, sub-redes, control plane e *node group*).
- `t3.medium` = 2 vCPU / 4 GiB. Para reduzir custo no lab, use `--nodes 1` (registre a escolha).

[EVIDÊNCIA: "EKS cluster is ready" no fim do `eksctl`]

8.4 Configurar o contexto do `kubectl`

```
aws eks update-kubeconfig --name biblioteca --region us-east-1
kubectl get nodes                # 2 nós em estado Ready
```

[EVIDÊNCIA: `kubectl get nodes`]

8.5 Implantar

```
kubectl create secret generic biblioteca-secret \
  --from-literal=jwt-secret="$(openssl rand -base64 32)"

kubectl apply -f k8s/base/deployment.yaml
kubectl apply -f k8s/managed/service-loadbalancer.yaml

kubectl rollout status deployment/biblioteca
kubectl get pods -o wide
```

8.6 Obter o endereço público e validar

```
kubectl get service biblioteca -w
# aguarde a coluna EXTERNAL-IP deixar de ser <pending> (~3-5 min)
# será um hostname do tipo alb2c3...elb.us-east-1.amazonaws.com
```

Acesse `http://<EXTERNAL-IP>/login`, entre com `admin` / `admin` e faça uma operação.

```
kubectl logs deployment/biblioteca --tail=50
```

[EVIDÊNCIA: get service com EXTERNAL-IP + tela da aplicação no domínio do ELB + login OK + logs sem erro]

Por padrão o `type: LoadBalancer` no EKS provisiona um **Classic Load Balancer** (via *cloud controller* interno). É suficiente para o lab. Em produção usaria-se o **AWS Load Balancer Controller** com NLB/ALB Ingress.

8.7 Exclusão — ver seção 15

9. Provedor 2 — Microsoft Azure (AKS)

Produto: Azure Kubernetes Service · **Região de exemplo:** `eastus`. Roteiro conforme slides 33–34.

9.1 Assinatura e grupo de recursos

```
az login
az account show --output table
az group create --name rg-biblioteca --location eastus
```

9.2 Azure Container Registry (ACR)

```
# o nome do ACR é global e único: 5-50 caracteres alfanuméricos minúsculos
az acr create --resource-group rg-biblioteca --name acrbibliotecaEQUIPEN --sku Basic
az acr login --name acrbibliotecaEQUIPEN

docker tag biblioteca:1.0 acrbibliotecaEQUIPEN.azurecr.io/biblioteca:1.0
docker push acrbibliotecaEQUIPEN.azurecr.io/biblioteca:1.0
```

Ajuste `image:` no `deployment.yaml` para `acrbibliotecaEQUIPEN.azurecr.io/biblioteca:1.0`.

[EVIDÊNCIA: `az acr repository list --name acrbibliotecaEQUIPEN`]

9.3 Criar o cluster AKS integrado ao ACR

```
az aks create \
  --resource-group rg-biblioteca \
  --name biblioteca \
  --node-count 1 \
  --node-vm-size Standard_B2s \
  --tier free \
  --attach-acr acrbibliotecaEQUIPEN \
  --generate-ssh-keys
```

- **~5–10 min.**
- `--tier free`: o **control plane não é cobrado** (sem SLA financeiro).
- `Standard_B2s` = 2 vCPU / 4 GiB. Se houver erro de cota, tente `Standard_DS2_v2`.
- `--attach-acr`: o kubelet passa a puxar imagens do ACR **sem** `imagePullSecret`.

9.4 Credenciais e implantação

```
az aks get-credentials --resource-group rg-biblioteca --name biblioteca
kubectl get nodes

kubectl create secret generic biblioteca-secret \
  --from-literal=jwt-secret="$(openssl rand -base64 32)"

kubectl apply -f k8s/base/deployment.yaml
kubectl apply -f k8s/managed/service-loadbalancer.yaml
kubectl rollout status deployment/biblioteca
```

9.5 Validar

```
kubectl get service biblioteca -w  
# EXTERNAL-IP é um IP público real (costuma sair em ~1-2 min)
```

Acesse `http://<EXTERNAL-IP>/login`, autentique e faça uma operação de escrita.

[EVIDÊNCIA: `get nodes + get service` com IP + tela da aplicação + login OK + logs]

9.6 Exclusão — ver seção 15

10. Provedor 3 — Hetzner Cloud (VPS + K3s)

Produto: Hetzner Cloud, servidor **CX22** (2 vCPU x86 / 4 GB / 40 GB) · **Local de exemplo:** Nuremberg (`nbg1`). Roteiro conforme slides 35–37.

10.1 Criar o servidor

Pelo Console: *Project* → *Add Server* → Ubuntu 24.04, tipo **CX22**, adicione sua **chave SSH**, crie.

Ou pela CLI `hcloud` :

```
hcloud context create biblioteca          # cola o API token do projeto  
hcloud ssh-key create --name minha-chave --public-key-from-file ~/.ssh/id_ed25519.pub  
hcloud server create --name biblioteca --type cx22 --image ubuntu-24.04 \  
  --ssh-key minha-chave --location nbg1  
hcloud server ip biblioteca              # anote o IP público
```

10.2 Firewall (slide 36)

Crie um **Firewall** no Console (ou `hcloud firewall`) e associe ao servidor, liberando **entrada TCP**:

Porta	Uso
22	SSH
80, 443	HTTP/HTTPS (Ingress/proxy reverso)
30080	teste via NodePort

10.3 Instalar o K3s

```
ssh root@IP_DA_VPS  
curl -sfL https://get.k3s.io | sh -  
k3s kubectl get nodes          # 1 nó "Ready" (control plane + worker no mesmo host)
```

O K3s já inclui **Traefik** (Ingress) e **ServiceLB/Klipper** (permite até `type: LoadBalancer` usando o IP do nó). O kubeconfig fica em `/etc/rancher/k3s/k3s.yaml`.

10.4 Levar os arquivos para a VPS (slide 36)

Opção A — clonar este repositório no servidor:

```
ssh root@IP_DA_VPS
git clone https://github.com/Renan0204/t3-biblioteca-multicloud.git
cd t3-biblioteca-multicloud
```

Opção B — copiar apenas a pasta `k8s/` da sua máquina:

```
scp -r k8s/ root@IP_DA_VPS:/root/biblioteca-multicloud/
```

10.5 Implantar

A imagem pública do Docker Hub é puxada diretamente pelo K3s (sem registro adicional). Ajuste `image:` no `deployment.yaml` para `docker.io/SEU_USUARIO DOCKERHUB/biblioteca:1.0`.

```
k3s kubectl create secret generic biblioteca-secret \
  --from-literal=jwt-secret="$(openssl rand -base64 32)"

k3s kubectl apply -f k8s/base/deployment.yaml
k3s kubectl apply -f k8s/vps/service-nodeport.yaml

k3s kubectl rollout status deployment/biblioteca
k3s kubectl get pods,svc
```

10.6 Validar

Acesse `http://IP_DA_VPS:30080/login`, autentique com `admin` / `admin` e faça uma operação.

```
k3s kubectl logs deployment/biblioteca --tail=50
```

[EVIDÊNCIA: `get nodes` + `get pods/svc` + tela da aplicação em IP:30080 + login OK + logs]

10.7 Acesso externo, HTTPS e portas (slide 36)

- **Como o acesso externo foi configurado:** `NodePort` 30080 + Firewall Hetzner liberando a porta.
- **Portas liberadas:** 22, 80, 443, 30080.
- **HTTPS:** com um nome DNS apontando para o IP da VPS, aplicar `k8s/vps/ingress-traefik.yaml` e instalar o **cert-manager** com um `ClusterIssuer` Let's Encrypt para emissão automática de certificado. Alternativa: proxy reverso Nginx/Caddy no host terminando TLS e encaminhando para o `NodePort`.

11. Evidências e testes

Os arquivos disponíveis estão reunidos no [índice de evidências com legendas](#), organizado por ambiente. As capturas de login e cadastro local foram obtidas com Java, sem Docker.

Capturar, **para cada um dos três ambientes** (slide 38), ocultando tokens, chaves, Account/Subscription IDs e dados de cobrança:

#	Evidência	Comando de apoio
1	Provedor, produto e região	painel do provedor
2	Cluster e nós disponíveis	<code>kubectl get nodes -o wide</code>
3	Imagem publicada no registro	painel do Docker Hub / ECR / ACR
4	<code>Deployment</code> disponível	<code>kubectl get deployment biblioteca</code>
5	Pod <code>Running</code> e <code>1/1</code> pronto	<code>kubectl get pods</code>
6	Mecanismo de exposição	<code>kubectl get service</code> (ou Ingress)
7	Aplicação acessível externamente	navegador na URL pública
8	Login realizado	tela após autenticar + 1 operação de escrita
9	Logs sem erro impeditivo	<code>kubectl logs deployment/biblioteca</code>
10	Procedimento de remoção executado	painel mostrando recursos removidos

Salve os arquivos em `docs/evidencias/` com nomes como `aws-05-pods.png`, `azure-07-app.png`, `hetzner-09-logs.png`.

12. Comparação de custos

12.1 Regras adotadas (slide 40)

Parâmetro	Valor
Data da consulta	DD/MM/AAAA (preencher no dia da entrega)
Moeda	USD (com nota de conversão para BRL)
Regiões	AWS <code>us-east-1</code> · Azure <code>eastus</code> · Hetzner <code>nbg1</code>
SO / arquitetura	Linux / x86_64
Nós	AWS 2 × <code>t3.medium</code> · Azure 1 × <code>Standard_B2s</code> · Hetzner 1 × <code>CX22</code>
Horas/mês	~730 h (uso contínuo)
Créditos gratuitos	não embutidos nos totais (listados à parte)

Valores **estimados** não substituem a fatura real (slide 40). Reconsulte os preços na data da entrega nas páginas oficiais (seção 19).

12.2 Tabela quantitativa (modelo do slide 42 — valores de exemplo)

Critério	AWS (EKS)	Azure (AKS)	Hetzner (K3s)
Serviço Kubernetes	Amazon EKS	Azure AKS (tier Free)	K3s em VPS CX22
Região e configuração	us-east-1 · 2 × t3.medium	eastus · 1 × Standard_B2s	nbg1 · 1 × CX22
Gestão do cluster / mês	~US\$ 73,00 (US\$ 0,10/h)	US\$ 0,00	US\$ 0,00
Computação / mês	~US\$ 60,00 (2 nós)	~US\$ 30,00 (1 nó)	incluído no CX22
Disco e armazenamento	~US\$ 3,00 (2 × 20 GB gp3)	~US\$ 0–5 (OS disk)	incluído (40 GB)
Load balancer / IP / rede	~US\$ 18,00 (Classic ELB)	~US\$ 18,00 (Standard LB + IP)	US\$ 0,00 (IP do nó / NodePort)
Registro de imagem	~US\$ 0,10 (ECR < 1 GB)	US\$ 5,00 (ACR Basic)	US\$ 0,00 (Docker Hub público)
Tráfego de saída (baixo)	~US\$ 1,00	~US\$ 1,00	incluído (20 TB)
Total mensal estimado	≈ US\$ 155	≈ US\$ 56	≈ US\$ 4,10 (€ 3,79)
Tempo aproximado de implantação	~15–20 min	~6–10 min	~3–5 min

Observações:

- Reduzindo a AWS para **1 × t3.medium**, o total cai para **≈ US\$ 125/mês**.

- Créditos gratuitos (à parte): **AWS** não oferece crédito de Kubernetes; **Azure** dá US\$ 200 por 30 dias para contas novas; **Hetzner** não oferece crédito, mas o preço absoluto já é baixo.
- **Custo por hora de laboratório** (criar, validar, coletar evidências e destruir em ~3 h): AWS ≈ US\$ 0,60 (só o control plane) + nós; Azure ≈ nós; Hetzner ≈ US\$ 0,02.

12.3 Componentes de custo considerados (slide 41)

Gestão do control plane · máquinas/computação · disco dos nós · registro de imagem · balanceador/IP público · tráfego de saída · DNS (não utilizado) · observabilidade (não incluída) · impostos (não incluídos) · **tempo operacional da equipe** (ver 13).

13. Comparação qualitativa — vantagens e desvantagens

13.1 Tabela (modelo do slide 43)

Critério	AWS (EKS)	Azure (AKS)	Hetzner (K3s)
Facilidade de configuração	Média — <code>eksctl</code> ajuda, mas há VPC/IAM e ~20 min	Alta — um comando, <code>--attach-acr</code>	Média — criar VPS, instalar K3s, firewall e exposição manual
Kubernetes gerenciado	Sim	Sim	Não (autogerenciado)
Escalabilidade	Alta — Cluster Autoscaler, muitos tipos de nó	Alta — autoscaler, <i>node pools</i>	Baixa/Média — manual; dá para adicionar <i>agents</i>
Integração com registro	Boa — ECR + IAM do nó automático	Excelente — <code>--attach-acr</code>	Manual — Docker Hub ou registro próprio
Observabilidade disponível	CloudWatch Container Insights (add-on)	Azure Monitor / Container Insights (add-on)	Nenhuma nativa — instalar Prometheus/Grafana
Responsabilidade operacional	Média — control plane gerenciado; nós/add-ons com a equipe	Média	Alta — SO, K3s, backup, segurança e disponibilidade com a equipe
Vantagens principais	Ecossistema amplo, IRSA, ALB/NLB, presença de mercado	Menor custo gerenciado (control plane grátis), integração Microsoft Entra, limpeza por <i>resource group</i>	Custo baixíssimo, controle total, simplicidade conceitual, tráfego generoso
Desvantagens principais	Mais caro, mais componentes cobrados, curva inicial maior	Dependência do ecossistema Azure, cota de VM pode barrar	Sem HA, sem gerenciamento, tudo é responsabilidade da equipe
Cenário recomendado	Empresa já em AWS, precisa de escala/HA e serviços AWS	Melhor custo-benefício gerenciado para equipe pequena	Laboratório, estudo, cargas pequenas e previsíveis

13.2 Vantagens e desvantagens esperadas (slide 45)

Kubernetes gerenciado (EKS/AKS): menos esforço no *control plane*; integração nativa com identidade, registro, rede e observabilidade; recursos de HA e autoscaling. Em troca: mais itens cobrados, curva de aprendizagem do provedor e dependência de serviços específicos.

VPS com K3s (Hetzner): maior controle e custo inicial muito menor. Em troca: a equipe assume segurança, atualização, backup e disponibilidade; sem HA do *control plane*.

14. Recomendação final

Para este trabalho (aplicação demonstrativa, H2 em memória, réplica única, sem persistência): **Hetzner Cloud + K3s**.

Justificativa:

- Custo **~40x menor** que a AWS e **~14x menor** que a Azure no cenário comparado.
- A capacidade do CX22 sobra para uma única réplica do Spring Boot.
- Os diferenciais dos serviços gerenciados (HA do *control plane*, autoscaling, add-ons) **não agregam** a uma aplicação que roda com 1 réplica e sem estado persistente. Pagar por eles aqui seria desperdício.

Se o projeto evoluísse para produção real (banco gerenciado, múltiplas réplicas, HA, CI/CD, observabilidade): **Microsoft Azure (AKS)** — *control plane* sem custo no tier Free, `--attach-acr`, limpeza simples por *resource group* e bom equilíbrio entre esforço operacional e consumo.

AWS (EKS) é a escolha quando a organização já opera em AWS ou precisa de integração profunda (IRSA, ALB Ingress, ecossistema de serviços) — aceitando o custo mais alto e a maior complexidade inicial.

15. Limpeza dos recursos

Excluir **depois** de coletar todas as evidências. Confirmar no painel que nada cobrado permaneceu (slides 54 e 62).

15.1 AWS

```
# 1) Remover o Service ANTES do cluster (libera o Load Balancer)
kubectl delete -f k8s/managed/service-loadbalancer.yaml
kubectl delete deployment biblioteca
kubectl delete secret biblioteca-secret

# 2) Cluster + node group (stack CloudFormation)
eksctl delete cluster --name biblioteca --region us-east-1

# 3) Registro de imagem
aws ecr delete-repository --repository-name biblioteca --region us-east-1 --force
```

Conferir no Console: **EKS** (sem cluster), **EC2** → **Load Balancers** (nenhum), **EC2** → **Volumes** (sem EBS órfão), **CloudFormation** (stacks removidas), **VPC** criada pelo `eksctl` removida.

15.2 Azure

```
az group delete --name rg-biblioteca --yes --no-wait
```

Um comando remove **cluster, ACR, Load Balancer, IP público e discos**. Conferir em *Resource groups* que `rg-biblioteca` desapareceu (e o *node resource group* `MC_rg-biblioteca_biblioteca_eastus`).

15.3 Hetzner

```
# No servidor: desinstalar o K3s
ssh root@IP_DA_VPS "/usr/local/bin/k3s-uninstall.sh"

# Da sua máquina: destruir o servidor e recursos associados
hcloud server delete biblioteca
hcloud firewall delete <nome-ou-id-do-firewall>
hcloud ssh-key delete minha-chave          # se não for reutilizar
```

Conferir no Console Hetzner: **Servers, Firewalls, Volumes, Floating IPs** e **Load Balancers** vazios. Verificar a aba **Usage/Billing**.

16. Erros comuns e diagnóstico

16.1 Comandos de diagnóstico (slide 56)

```
kubectl get all
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl describe deployment biblioteca
kubectl describe pod <NOME_DO_POD>
kubectl logs deployment/biblioteca
kubectl logs deployment/biblioteca --previous
kubectl rollout status deployment/biblioteca
kubectl get endpoints biblioteca
```

16.2 Problemas mais prováveis (slide 55) e soluções

Sintoma	Causa provável	Solução
<code>exec format error</code> no log do Pod	imagem <code>arm64</code> em nó <code>amd64</code>	rebuild com <code>docker buildx build --platform linux/amd64</code>
<code>ImagePullBackOff</code> / <code>ErrImagePull</code>	registro privado sem autorização	tornar o repositório público, ou <code>--attach-acr</code> (Azure), ou <code>imagePullSecret</code>
<code>CrashLoopBackOff</code> + <code>OOMKilled</code> em <code>describe pod</code>	memória insuficiente	subir <code>resources.limits.memory</code> para <code>1Gi</code>
Pod reinicia durante o boot	<code>livenessProbe</code> cedo demais	já usamos <code>startupProbe</code> ; aumentar <code>failureThreshold</code> se o host for lento
<code>CreateContainerConfigError</code>	<code>Secret</code> ausente ou chave diferente de <code>jwt-secret</code>	criar <code>biblioteca-secret</code> antes do <code>Deployment</code>
<code>Service</code> sem <code>EXTERNAL-IP</code> (AWS) por minutos	provisionamento do ELB	aguardar 3–5 min; checar <code>kubectl describe svc biblioteca</code>
<code>EXTERNAL-IP</code> eterno <code><pending></code> (VPS)	sem <i>cloud controller</i>	usar <code>NodePort</code> (feito) ou o Klipper do K3s
Página não abre em <code>IP:30080</code> (Hetzner)	firewall bloqueando a porta	liberar TCP 30080 no Firewall Hetzner
<code>Service</code> sem <i>endpoints</i>	<code>selector</code> ≠ <code>labels</code> do Pod	ambos devem ser <code>app: biblioteca</code>
Conexão na porta errada	<code>targetPort</code> incorreto	<code>targetPort: 8080</code> (porta do container)

16.3 Dois problemas para relatar no documento (slide 56)

Descrever no relatório **pelo menos dois** problemas realmente enfrentados pela equipe, como foram investigados (qual comando revelou a causa) e como foram resolvidos.

17. Checklists finais

17.1 Checklist técnico (slide 57)

- ☐ A imagem usa **Java 21**.
- ☐ O build Maven terminou sem erros.
- ☐ A aplicação funciona localmente em container (`http://localhost:8080/login`).
- ☐ O segredo JWT é fornecido **externamente** (`Secret` → env), nunca no repositório.
- ☐ O `Deployment` usa **1 réplica** (H2 em memória).
- ☐ O Pod fica `1/1 Running` .
- ☐ O `Service` encaminha a porta pública para **8080**.
- ☐ A aplicação é acessível **externamente** nos três ambientes.
- ☐ Login + **uma operação de escrita** validados nos três ambientes.
- ☐ Existe procedimento de **limpeza** para cada provedor (seção 15).

17.2 Checklist do e-mail de entrega (slides 49–50)

- ☐ Destinatário: **diogo.p.ranghetti@gmail.com**.
- ☐ Assunto: `[Cloud DevOps] T3 - Biblioteca em Kubernetes` (a equipe não tem número; identificar pelos integrantes no corpo).
- ☐ Corpo lista **todos os integrantes**.
- ☐ **PDF** anexado e legível.
- ☐ Arquivos editáveis ou **link do repositório** com acesso ao professor.
- ☐ **Nenhuma** credencial/segredo real enviada ou visível em prints.
- ☐ Tabela **ou** gráfico comparativo de custos presente no documento.
- ☐ Os **três provedores** claramente identificados (AWS, Azure, Hetzner).
- ☐ Enviado **antes do encerramento da aula**.

17.3 Entregáveis (slide 46)

Documento PDF · link do repositório · `Dockerfile` e `.dockerignore` · manifestos Kubernetes · tutorial dos três provedores · evidências · tabela/gráfico de custos · comparação de vantagens/desvantagens · recomendação final justificada · identificação dos integrantes.

18. Glossário

Termo	Significado
Imagem	Pacote imutável com aplicação, runtime e dependências
Container	Instância em execução de uma imagem
Registro	Serviço que armazena e distribui imagens (Docker Hub, ECR, ACR)
Cluster	Conjunto de recursos que executa workloads Kubernetes
Node	Máquina que executa Pods
Pod	Menor unidade implantável do Kubernetes
Deployment	Recurso que mantém e atualiza um conjunto de Pods
Service	Endpoint estável que encaminha tráfego para Pods selecionados
EKS / AKS	Kubernetes gerenciado da AWS / da Azure
K3s	Distribuição Kubernetes leve, de nó único neste trabalho
VPS	Servidor virtual privado administrado pelo cliente
Control plane	Componentes que coordenam o estado do cluster
Load balancer	Recurso que distribui tráfego e fornece acesso externo
Egress	Tráfego de dados que sai do provedor/região
NodePort	Porta fixa aberta em todos os nós para expor um Service
Ingress	Regra de roteamento HTTP(S) para Services, via <i>controller</i> (Traefik no K3s)

19. Referências

Aplicação e tecnologias

- RANGHETTI, Diogo P. *Biblioteca*. GitHub. <https://github.com/dpRanghetti/biblioteca>
- Docker. *Multi-stage builds*. <https://docs.docker.com/build/building/multi-stage/>
- Docker. *Building best practices*. <https://docs.docker.com/build/building/best-practices/>
- Kubernetes. *Deployments*. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- Kubernetes. *Services, Load Balancing, and Networking*. <https://kubernetes.io/docs/concepts/services-networking/>
- K3s. *Quick-Start Guide*. <https://docs.k3s.io/quick-start>

Provedores e custos

- AWS. *Getting started with Amazon EKS — eksctl*. <https://docs.aws.amazon.com/eks/latest/userguide/getting-started-eksctl.html>
- AWS. *Amazon EKS Pricing*. <https://aws.amazon.com/eks/pricing/>

- AWS. *Amazon EC2 On-Demand Pricing*. <https://aws.amazon.com/ec2/pricing/on-demand/>
 - Microsoft. *Quickstart: Deploy an AKS cluster using Azure CLI*. <https://learn.microsoft.com/azure/aks/learn/quick-kubernetes-deploy-cli>
 - Microsoft. *Azure Kubernetes Service (AKS) Pricing*. <https://azure.microsoft.com/pricing/details/kubernetes-service/>
 - Microsoft. *Azure Container Registry Pricing*. <https://azure.microsoft.com/pricing/details/container-registry/>
 - Hetzner. *Cloud — Pricing*. <https://www.hetzner.com/cloud/>
 - Hetzner Docs. *Install and configure K3s / kubectl*. <https://community.hetzner.com/tutorials/>
 - Rancher / SUSE. *K3s Documentation*. <https://docs.k3s.io/>
-

Documento produzido para o Trabalho 3 da disciplina Cloud Computing e DevOps Avançado.